



Rapport IA

Groupe 1 : Onitama



Membres du groupe:

1. Ifrel Rinel MAKOUNDIKA KIDZOUNOU
2. Arthur D'HERIN
3. Santiago ROQUE
4. Kevin NTYAM
5. Guillaume LAUSSAC
6. Akin KUTLU
7. Akram DABOUSSI

Tutrice de Projet : Dima Dalloul

Sommaire

Sommaire	2
Description du jeu	3
Pistes de développement et réflexions	3
Mise en Oeuvre	4
Structure	4
Evaluation	5
Optimisations	6
Difficulté des IA	6
Tests et statistiques	7
Conclusion	8

Description du jeu

Onitama est un jeu où deux joueurs s'affrontent avec 5 pions chacun, 1 pion Maître et 4 pions Élèves sur un plateau de 5x5 cases.

Chaque Joueur possède 2 cartes tirées aléatoirement au début de la partie.

Une carte supplémentaire qui sera échangée avec la dernière carte jouée à chaque tour est également présente sur le plateau, elle n'est pas jouable avant d'être récupérée par un joueur.

Toutes les cartes sont uniques.

Les pions de tous types peuvent se déplacer conformément aux déplacements indiqués sur les cartes, la case noire sur la carte choisie indique la position du pion qu'on souhaite déplacer.

Tous les déplacements (cases colorées non noires) sur la carte sont valides à condition de ne pas écraser ses propres pions ou sortir du terrain.

Pour gagner, il faut prendre le pion maître de l'adversaire ou placer son propre pion maître à l'entrée du temple adverse (position initiale du pion maître adverse en début de partie).

Une partie dure en moyenne entre 10 et 15 minutes.

Pistes de développement et réflexions

Le but est de développer une IA pouvant jouer un coup correspondant à un niveau donné (FAIBLE, MOYEN, FORT).

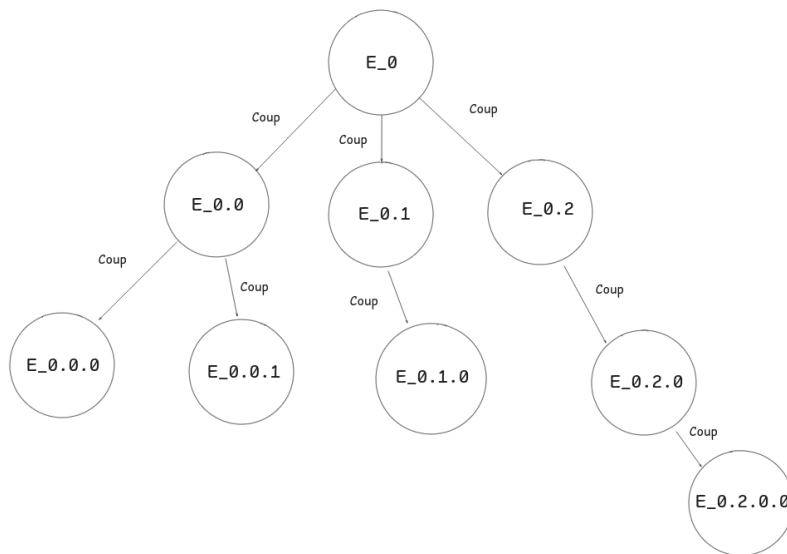
Pour ce faire, il s'agit de permettre à l'IA de savoir quel va être l'impact d'un coup sur le terrain et à quel point son effet sera positif ou négatif. Plus le nombre de coups évalués par l'IA en avance est élevé, plus le coup choisi aura un effet positif pour l'IA.

En effet, comme aux échecs, ce qui fait la différence c'est la capacité de se projeter. On souhaite également faire en sorte que l'IA ne soit pas imbattable pour laisser une chance au joueur Humain de gagner.

Mise en Oeuvre

Structure

Pour permettre à l'IA de calculer plusieurs coups à l'avance, on utilise un arbre pour stocker l'arborescence d'états du jeu, chaque nœud représentant un nouvel état du jeu obtenu en jouant un coup depuis l'état parent.

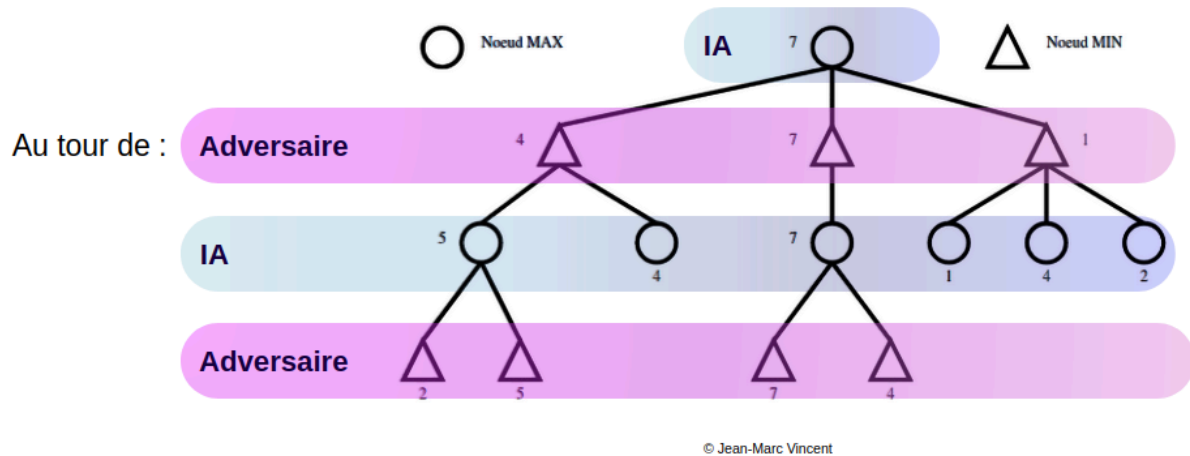


(Succession d'états du jeu sous forme arborescente)

Une fois qu'on arrive à générer une succession d'état en maintenant une relation hiérarchique cohérente, on peut donner une valeur à chaque configuration, donc chaque nœud, indiquant si l'état est intéressant dans l'immédiat ou pour la suite de la partie.

Pour cela, nous utilisons l'algorithme MiniMax, on considère que l'IA va jouer le coup l'avantageant le plus (donc qu'elle va maximiser son gain) et que son adversaire va jouer parfaitement (minimisant ainsi son gain).

Quand c'est au tour de l'IA de jouer, on se situe sur un nœud MAX qui va sélectionner la meilleure configuration possible de toutes les configurations suivantes possibles, donc le nœud avec la meilleure valeur. Quand c'est au tour de l'adversaire, on se situe sur un nœud MIN qui va récupérer la plus petite valeur obtenue parmi tous les états suivants possibles, pour simuler un jeu parfait, mettant l'IA en difficulté autant que possible. Pour simuler ces parties possibles, on part des états feuilles qu'on évalue en leur donnant une valeur, puis on remonte l'arbre en suivant les principes ci-dessus, jusqu'à atteindre la racine, à partir de là, on peut choisir le meilleur coup à jouer.



(Arbre MiniMax annoté pour mettre en évidence qui joue)

Evaluation

Pour être capable de choisir en alternant la pire et la meilleure valeur un coup, il faut être capable de donner une valeur arbitraire à un état du jeu. On pourrait par exemple donner +1 comme valeur à une configuration gagnante pour l'IA et -1 pour une configuration perdante, l'inconvénient est qu'il faudrait descendre très profondément dans l'arbre.

En sachant que la complexité d'un tel parcours est $O(b^p)$ avec b le facteur de branchement, soit le nombre d'états successeurs d'un état courant, et p la profondeur, soit le nombre de coups pris en compte :

Profondeur /Facteur de branchement	Facteur de branchement en moyenne ($b = 15$)	Facteur de branchement dans le pire des cas ($b = 40$) (rare)
2	225	1 600
3	3 375	64 000
4	50 625	2 560 000
5	759 375	102 400 000
6	11 390 625	4 096 000 000
7	170 859 375	163 840 000 000
...	...	...

Le nombre d'états à évaluer augmente de plus en plus avec la profondeur.

On doit donc se limiter à une certaine profondeur pour évaluer le moins d'état possible pour limiter le temps de calcul. On ne peut donc plus savoir en empruntant un chemin si il mène obligatoirement à la victoire, on utilise donc une heuristique pour évaluer de manière arbitraire une configuration donnée.

L'heuristique utilisée est une somme pondérée de sous-heuristiques :

```
heuristique(etatJeu) =      k1 * nombrePions(etatJeu)
                          +   k2 * valeurCartes(etatJeu)
                          +   k3 * nombreSuccesseurs(etatJeu)
                          +   k4 * distancePionsCourantMaitreAdverse(etatJeu)
                          +   k5 * distancePionsAdverseMaitreCourant(etatJeu)
                          +   k6 * distanceMaitreAdverseTempleCourant(etatJeu)
                          +   k7 * distanceMaitreCourantTempleAdverse(etatJeu)
                          + victoireDefaite(etatJeu)
```

Où victoireDefaite() est une heuristique qui prend le dessus sur toutes les autres et où k_i avec $1 \leq i \leq 7$ est un coefficient pour pondérer une heuristique. Il est possible d'entraîner deux IA l'une contre l'autre avec des valeurs de coefficient aléatoires ou automatisées pour déterminer quelle combinaison de coefficients mène le plus souvent à la victoire.

Optimisations

Comme vu précédemment, le calcul en profondeur est très coûteux, c'est pourquoi nous utilisons l'élagage Alpha-Beta qui permet de ne pas chercher plus loin quand on obtient une valeur qui est soit plus grande qu'un noeud MIN ancêtre (coupure beta), soit plus petite qu'un noeud MAX ancêtre (coupure alpha), il s'agit de pas effectuer des calculs en plus quand le noeud supérieur ne conservera pas de toute façon les valeurs suivantes. On peut donc se permettre de parcourir un peu plus profondément l'arbre.

Difficulté des IA

Les trois niveaux d'IA implémentés utilisent tous l'arbre MiniMax, seule la profondeur varie. En effet, la qualité d'un coup est influencée par la capacité de prévoir un certain nombre de coups.

Tests et statistiques

Affrontements entre deux IA de niveaux différents

Test de la fonction de construction		
	IA Faible	IA Moyenne
Type d'heuristique	<i>MiniMax</i>	<i>MiniMax</i>
Évaluation	<i>Avantage, pénalité</i>	<i>Avantage, pénalité</i>
Premier joueur	<i>Aléatoire</i>	
Nombre de parties	<i>100</i>	
Nombre de victoires	<i>9</i>	<i>91</i>
% de victoires	<i>9 %</i>	<i>91 %</i>

Test de la fonction de construction		
	IA Moyenne	IA Forte
Type d'heuristique	<i>MiniMax</i>	<i>MiniMax</i>
Évaluation	<i>Avantage, pénalité</i>	<i>Avantage, pénalité</i>
Premier joueur	<i>Aléatoire</i>	
Nombre de parties	<i>100</i>	
Nombre de victoires	<i>13</i>	<i>87</i>
% de victoires	<i>13 %</i>	<i>87 %</i>

On observe que l'IA Faible est battue dans la majorité des cas par l'IA Moyenne, qui elle-même est battue dans la majorité des cas par l'IA Forte.

Conclusion

Après observations, la combinaison de différentes optimisations comme l'élagage Alpha-Beta et le parcours avec Horizon Fini, avec une heuristique pertinente, permet d'obtenir une IA efficace en trois niveaux qui a un temps de calcul acceptable pour garantir une bonne expérience utilisateur.

On remarque aussi que la profondeur est un paramètre décisif qui influence énormément la qualité du coup choisi et qu'elle permet de définir des niveaux de difficultés pertinents pour un joueur humain.